

A-OSP

Core Idea & Proof Mechanics Brief

How A-OSP turns AI-assisted work into typed, local, receipt-backed and governable evidence.



*A-OSP is not a better answer generator. It is a proof machine around AI work.
It does not open the black box. It moves the trust boundary outside the model.*

Companion brief — claims anchored to live repo [main@e492290b](#) · June 2026.
Deep layers live in the Whitepaper, the Technical Due Diligence and the ROA corpus; this brief points into them instead of repeating them.

Gianluca Conte · contegianluca@hotmail.com · 2026 · confidential discussion draft · not legal or financial advice

What this brief proves — and what it does not

THIS BRIEF CLAIMS

- the mechanics of operational reconstructability: action → stripe → readback/receipt → scoped recall → artifact reuse
- Data intake atomization (D1) seam materially advanced on main, with negative tests
- every claim anchored to a repo path or a corpus section

IT DOES NOT CLAIM

- semantic truth, legal certification, production readiness
- full D1 / EQL (epistemic query language) proof-grade before C1 convergence artifacts close
- that fluent output, confidence, logs or UI green are proof

CLAIM RULE — every technical statement in this brief carries exactly one status:

AS-IS

HARDENING

CANDIDATE

TARGET

HYPOTHESIS

NON-GOAL

WHERE THE DEPTH LIVES

Whitepaper (WP)

A-OSP architecture, primitives, proof contracts

Technical Due Diligence (TechDD)

A-OSP live repo evidence, gaps, gates, metrics

ROA corpus + annexes

theory: RLA · CRC · ECNN foundations

Every Map Leaves Something Out

humanistic entry point, why horizons exist

Whole corpus: <https://github.com/Luke883i/ROA>

PRIMITIVES GLOSSARY

Stripe / .aosp.txt

append-only typed text record; durable knowledge lives here

Readback

read-after-write verification with canonical hash comparison

EQL

epistemic query language for scoped, stripe-shaped retrieval

MONL

operator layer / transaction membrane over parsed stripes

Authority Matrix

doc/generated/code authority classification, anti-drift

Artifact Witness

dependency map: sources, gaps, unsupported claims

Receipt

persisted proof object emitted after readback/compare

D1–D5

intake · enrichment · links · scoring · kernel/artifact phases

FAE

Focus–Attention–Execution envelope for bounded model calls

Error Truth

canonical failure envelope and capability state

Proof Spine

target composition of receipts/witnesses across phases

Governance Run

human review context and decision object

A-OSP makes model output usable where evidence matters

Without A-OSP

AI work is fluent but hard to reconstruct. Evidence, gaps, source state and review boundaries must be recovered after the fact — often during audit.

With A-OSP

AI work becomes local, typed, queryable, receipt-backed and governable. Non-knowledge becomes a first-class state, not an embarrassment.

The category

Proof infrastructure for AI-assisted work: the technical bridge behind partner collaboration, anchor formation and CTO diligence.

WHY IT COMPOUNDS — A-OSP gains value from the forces that make prompt-first AI fragile

Trust

every receipt and remediation adds proof capital

Knowledge

typed reticulum cuts rework and reconstruction cost

Regulation

new rule → schema → evidence workflow → demand

Model optionality

new models become processors, not threats

Antifragile QA

hallucination → debt → negative test → guard

Good switching cost

the customer owns the proof graph; lock-in is asset-based

DEFINITION

What A-OSP is — and is not

A-OSP IS

- a browser-native epistemic operating environment
- a text-first virtual epistemic filesystem
- a proof-and-knowledge substrate
- a bounded model-call orchestration layer
- a human-facing diagnostic workstation

A-OSP IS NOT

- a chatbot or document generator
- a GRC dashboard or model-provider memory
- arbitrary folder storage
- legal certification
- an autonomous governance engine

Engineering interpretation: “ontological” means work becomes typed entities and relations; “epistemic” means those entities carry schema, lineage, proof state and governance status.

Proof is a chain, not a badge

A proof-sensitive transition is green only when the whole chain holds:



FORBIDDEN EQUIVALENCES



What a receipt-backed state proves — and does not: it proves persistence, scope and reconstruction boundaries. It does not prove semantic truth or legal sufficiency. For any artifact, the system must answer which sources were used, what was inferred, what is missing, what failed and who reviewed — from persisted objects, not from memory.

False-green prevention is not a UI detail; it is the bridge between engineering truth and governance truth.

Truth does not live in the model, UI, DB or export

.aosp.txt / typed stripes

durable source of truth — portable, diffable, recoverable

Directus / PostgreSQL

cache, index, admin — rebuildable, non-canonical

React UI / app lenses

projection & workspace — must not own proof identity

LLM provider

replaceable processor — bounded call, no durable truth

PDF / DOCX / JSON export

downstream representation — not witness unless proof-bound

Receipt / readback / witness

proof surface — explicit state, lineage, hashes

Anti-lock-in contract: only the text-first substrate owns durable knowledge; everything else is cache, projection, processor, export or proof surface.

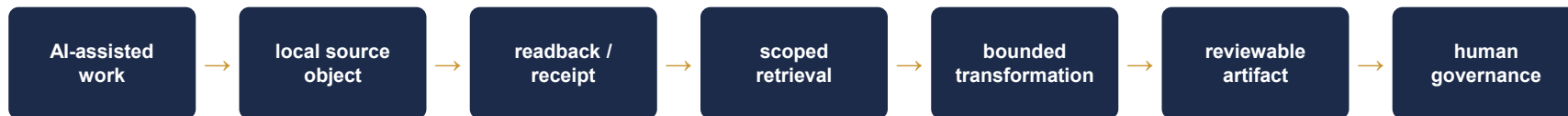
Browser runtime + semantic filesystem + control plane

Browser runtime	sandbox · identity · permissions · HTTP/WS · local storage
Frontend shell	React/Vite · routes · app lenses · workstation
State + access	hooks · React Query · useFilesystem · DataStore
Backend control	parser-service APIs · EQL · MONL · diagnostics
Durable source	.aosp.txt stripes · schema registry · append/readback
Diagnostics & AI	WebSocket · Observatory · Error Truth · provider adapters
Operations	CI/CD · guards · generated docs · authority checks

Infrastructure adequacy rule: a technical review must see runtime, shell, state layer, parser-service, source/cache split, diagnostics, AI adapters, CI guards and recovery path.

The single mechanics line

Every workflow must be reducible to one inspectable chain:



Mechanical thesis

The model is not the source of truth. The model is a replaceable processor over a local proof substrate.

Proof thesis

A proof-sensitive transition must persist, read back, compare, emit a receipt and expose downstream eligibility.

The answer is not the product. The path, proof state and review boundary are the product.

A-OSP turns work into typed epistemic objects

answer_ source capture	atom_ candidate fact	atom_validation_ human gate	evidence_gap_ missing support
enrichment_ added context	link_ relation	score_ weighted evaluation	kernel_ compact state
artifact_ review output	receipt_ proof object	witness_ dependency map	governance_run_ human decision

The object grammar is the moat: once AI work is typed, it can be queried, inspected, tested, promoted, rolled back and governed.

Intake becomes proof-sensitive only with receipt + readback

D1 converts raw human/domain material into persisted, typed, validated atoms. It is not proof-sensitive until it persists, reads back, compares and emits a receipt.



REPO-MINED D1 PROOF SEAM

POST /api/v1/proof/d1/append

single proof-grade producer · fail-closed ProblemDetail
returns receipt + proofSessionContext

PASS only after readback / hash confirmation

AS-IS · MATERIALLY ADVANCED

Route, service, readback, receipt and negative tests live on main. Full D1/EQL proof-grade still gated by EQL/C1 closure.

Anchor: routes/proof-d1.ts · services/d1-proof-append-service.ts · test/proof-d1.test.js · server.ts (full paths → slide A2)

Generic append may create canonical data; only this seam can produce proof PASS.

EQ · FAE · MONL — what a model call may see, do and become

EQ

what a call may SEE

Scoped, session-bound retrieval over append-only stripes with proof metadata (queryHash, resultHash, EQQueryReceipt). Not SQL-with-extras.

FOUNDATIONS AS-IS · PROOF =
CANDIDATE

FAE

what a call may DO

Focus (which objects, via EQ) · Attention (min confidence, max atoms, forbidden actions) · Execution (model, schema, fallback). The LLM is a processor, never memory or authority.

PROFILE · CONSUMED BY PROOF GATES

MONL

what a transform may BECOME

Transaction membrane between “we can transform this” and “this is proof-eligible downstream”. Never silently mutates canonical proof state.

TARGET · KEPT APART FROM D1/EQ

CANONICAL FAE STRIPE PROFILE

```
fae_[phase:D1]
  focus: EQ SELECT answer_ WHERE session=$SES
  attention: min_conf=0.65, max_atoms=10
  execution: model, temp, schema, fallback=error
```

EQ tiers: Core (read-only retrieval) · Proof (canonical scope + receipts) · Analytic (aggregation, lineage).

Authority Matrix: classifies doc / generated / code authority — anti-drift governance when sources conflict.

No raw LLM authority: EQ-selected objects → FAE-bounded call → MONL-controlled transform → stripe, or Error Truth.

Failure must become diagnosable, not cosmetic



False-green blockers

- UI completion cannot become proof
- fallback cannot manufacture proof
- degraded capability must be visible
- out-of-scope → review_required / proof_ineligible

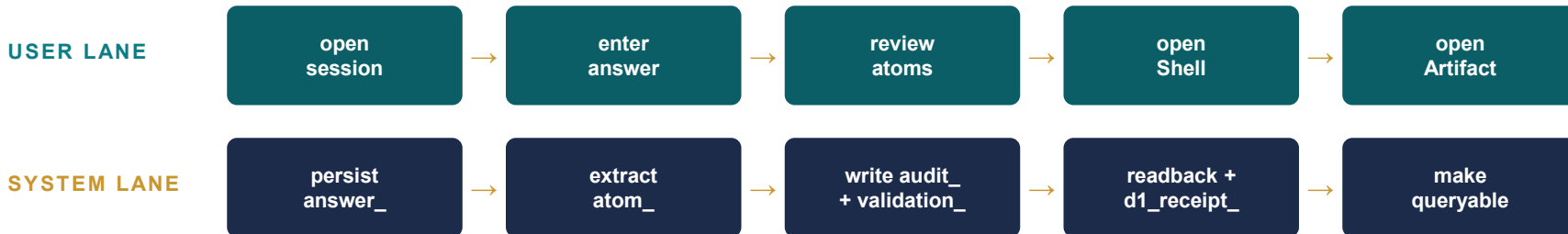
Canonical payload

source surface · phase · cause · severity ·
remediation · capability state · proof
eligibility · trace id

D1 already fail-closed: RFC 7807 ProblemDetail with explicit FAIL receipt.

Without Error Truth, every proof failure becomes a UX negotiation. With Error Truth, every failure becomes an object.

From a human answer to a persistent, queryable stripe



```
session_ | id:SES-001 | horizon:D1-intake | status:open → atom_001 | type:risk | session:SES-001 | validation:accepted  
| proof:receipt-backed
```

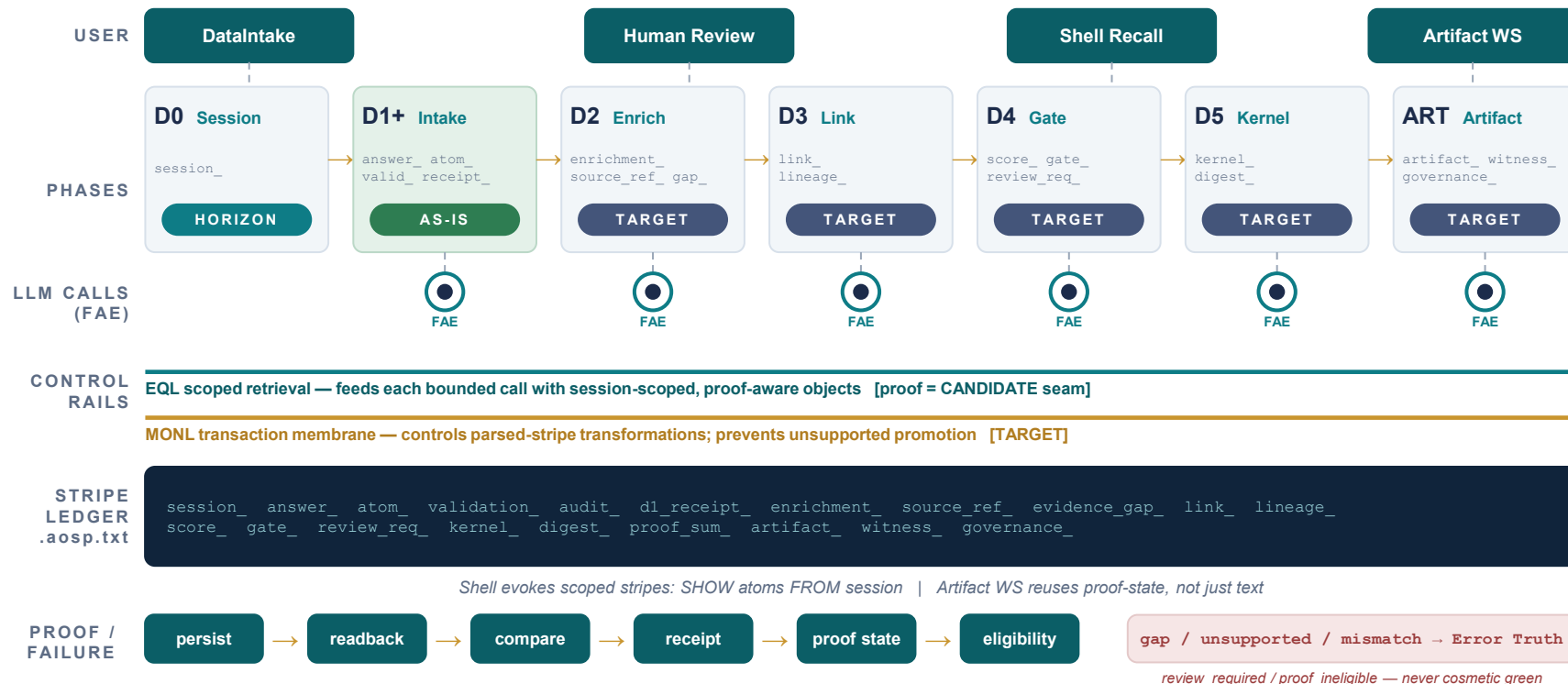
A session is a horizon boundary: it makes later retrieval scoped instead of ambient.

Validation writes audit state and blocks downstream authoritative use while debt remains. A generated object without readback is not D1 proof.

The product value is not the generated text. It is the reconstructable path from user input to durable proof state.

E2E PROOF CONVEYOR — FULL FLOWCHART

LLM is a processor · stripe ledger is memory · receipts + review state make work usable



Shell recall & Artifact Workstation reuse proof state, not text

Shell recall

The user evokes D1 objects from scoped stripes — atoms, validation state, receipt metadata — via EQL-style retrieval. The ledger is not a dead log; it is an evocable working memory.

```
SHOW atoms FROM session SES-001
PROOF SELECT atom_ WHERE session="SES-001"
  WITH validation, receipt
```

AS-IS / HARDENING

Artifact Workstation

Assembles a reviewable artifact from scoped objects — not prose generation. Sections carry explicit status: supported · derived · gap · review-required. Unsupported claims become visible, not buried.

Downstream: ArtifactWitness · Proof Spine · GovernanceRun

WITNESS / SPINE = TARGET

Artifact output is trustworthy only when it preserves proof status, gaps and review state.

Claim · repo anchor · posture, in one table

D1 proof route + producer	routes/proof-d1.ts · services/d1-proof-append-service.ts	AS-IS · ADVANCED
D1 negative tests (false-green guards)	test/proof-d1.test.js — FAIL never PASS on readback/hash mismatch	LANDED
D1 UI honesty / fail-closed intake	DataIntake proof view model · CompletionModal D2 gate	AS-IS · HARDENING
EQL proof envelope + unified route	EQLRunResult · EQLQueryReceipt · canonical hashes (issues #3425-#3488)	CANDIDATE · SEAM
C1 convergence artifacts	generated report · scans · negative-test results	TARGET · PROOF GATE
ArtifactWitness · Proof Spine · GovernanceRun	downstream composition	TARGET
Enterprise deployment · legal certification	—	NON-GOAL UNTIL PILOTS
Diligence upgrade before external CTO circulation: attach commit snapshot (main@e492290b · 2026-06), test command + pass/fail output, line-level excerpts. Repo-referenced now; diligence-grade once attached.		

Six parent metrics with formulas — not vanity metrics

`d1_receipt_pass_rate`

successful D1 receipts / append attempts

INSTRUMENTED NOW

`readback_success_rate`

readback matches / persisted writes

INSTRUMENTED NOW

`fae_bounded_call_rate`

bounded LLM calls / all LLM calls

HARNESS TARGET

`shell_recall_success_rate`

scoped recalls / recall attempts

HARNESS TARGET

`artifact_reuse_coverage`

artifact sections with source objects / all sections

PILOT METRIC

`unsupported_claim_detection_rate`

detected unsupported claims / seeded unsupported claims

BENCHMARK

C1 gate zeros: `d1_pass_without_readback = 0 · eq1_success_without_receipt = 0 · d2_handoff_without_receipt = 0 · query_hash_reproducibility = 100%`

A-OSP succeeds only if proof, query, honesty and operations metrics make reconstructability observable.

Three gates, in order — nothing skips the proof gate

1 · C1 closure

EQL proof envelope, unified route, canonical hashes, negative-test report. Only then: “D1/EQL proof-grade”.

PROOF GATE

2 · P1 enterprise pilot

Security posture, recovery, observability, provider exposure, runbooks — honesty metrics under real workload.

PILOT GATE

3 · Anchor formation

Partner collaboration and structure: see the companion AFM — Anchor Formation Memorandum.

PARTNER GATE

The answer is not the product. The path, proof state and review boundary are the product.

Read next: TDD v1 for live evidence · Whitepaper for architecture · AFM for the partnership frame.

Worked example — stripe set, API payload, negative tests

ILLUSTRATIVE STRIPE SET

```

session_ | SES-001 | proof:pending
answer_001 | session:SES-001
  | source:DataIntake | proof:pending
atom_001 | type:risk
  | validation:accepted
audit_001 | actor:human
  | result:accepted
dl_receipt_001 | readback:pass
  | hash:... | proof:eligible

```

API SEAM

```

POST /api/v1/proof/dl/append
{ sessionId, answerId,
  idempotencyToken,
  validation:{actor,status} }

200 → { status:"PASS",
  receiptId, readback:{hash},
  proofSessionContext }

failure → problem+json, 422
with explicit FAIL receipt

```

NEGATIVE-TEST CONTRACT

```

proof-dl.test.js

happy path persists dl_receipt_
idempotency token deterministic
failed readback => FAIL
append error      => FAIL
hash mismatch    => never PASS
no UI completion may
  masquerade as proof

```

Full anchors: backend/parser-service/src/routes/proof-dl.ts · src/services/dl-proof-append-service.ts · src/server.ts · test/proof-dl.test.js

Proof bundle target & definition of done

TARGET PROOF / ECONOMIC BUNDLE

```
{
  "proof_bundle_id": "proof.session.SES-001",
  "source_objects_count": 42,
  "receipt_count": 11,
  "readback_count": 11,
  "lineage_edges": 38,
  "unsupported_claims_count": 2,
  "missing_evidence_count": 3,
  "reused_context_objects": 31,
  "new_llm_calls": 3,
  "status": "review_required"
}
```

SYSTEM DoD (compressed)

- every LLM call inside a FAE envelope, fed by EQL scope
- D1 green requires persist → readback → compare → d1_receipt_
- failure becomes Error Truth, never cosmetic green

DoD OF THIS BRIEF

- a CTO understands where truth lives and what the D1 happy path is
- AS-IS / CANDIDATE / TARGET / NON-GOAL never collapse into one claim
- every deep question has a PDF pointer (→ A4)

Bundle families: Repository · Schema · D1Proof · EQLQuery · Pipeline · Artifact · Governance · ErrorTruth · ProviderExposure

Source map — which document answers which question

Why does every map need a declared horizon? Every Map Leaves Something Out — humanistic / philosopher entry point

What is the full theory behind the architecture? ROA paper · RLA-CRC-ECNN main paper + Annexes A–G

How does this relate to Wolfram's Ruliad? AI as Observer Compiler (parts 1–2)

How is the platform architected end to end? [WP] A-OSP Webapp (Augmented Ontological Semantic Platform) WHITEPAPER

What does the live repo actually prove today? [TechDD] A-OSP Technical Due Diligence v1 — Infrastructure, Runtime, Topology

What is the partnership / formation frame? AFM — Anchor Formation Memorandum (companion to this brief)

Whole corpus: <https://github.com/Luke883i/ROA>